

Rapport technique — Laboratoire 3

Informations générales

- Cours : Services Web 1 (420-941-MA)
 - Session : Hiver 2026
 - Projet : Nomina (Parcours A — Backend Node.js + Frontend React)
 - Auteur : **[À compléter]**
 - Date : 22 février 2026
-

1) Contexte et objectifs

Le Laboratoire 3 complète le backend réalisé au Laboratoire 2 par :

1. une interface frontend React moderne,
2. un système d'authentification sécurisé,
3. l'intégration complète frontend-backend.

L'objectif du projet Nomina est de proposer une application full-stack de génération et de gestion de contenus narratifs (personnages, lieux, cultures, catégories, concepts, etc.), avec accès protégé selon l'authentification et les droits d'administration.

2) Architecture technique

2.1 Vue d'ensemble

Architecture client-serveur en 3 couches :

- **Frontend** : React + Vite + TypeScript
- **Backend API** : Node.js + Express + TypeScript
- **Données** : PostgreSQL via Prisma ORM

Flux principal :

1. le frontend envoie des requêtes HTTP (**fetch**) vers l'API,
2. le backend applique validation, auth et logique métier,
3. Prisma exécute les opérations en base,
4. la réponse JSON revient au frontend.

2.2 Structure du projet

- **Nomina-frontend/** : pages React, composants UI, service API, routing
- **Nomina-backend/** : routes Express, controllers, middleware auth, services, Prisma
- **README.md** (racine) : démarrage global du monorepo

2.3 Choix techniques

- **React + Vite** : rapidité de développement et build performant.
 - **Express** : API REST simple et lisible.
 - **Prisma** : typage fort, migrations, productivité ORM.
 - **Clerk** : gestion professionnelle des sessions et tokens.
-

3) Authentification et sécurité

3.1 Solution retenue

La solution choisie est **Clerk** (option recommandée). Elle est intégrée côté frontend et validée côté backend.

3.2 Implémentation

- **Frontend** :
 - `ClerkProvider` au niveau de l'application,
 - récupération de token via `useAuth()`,
 - protection des routes sensibles (`dashboard`, `admin`).
- **Backend** :
 - middleware `requireAuth` qui valide le bearer token,
 - middleware `requireAdmin` pour les routes administrateur.

3.3 Mesures de sécurité

- vérification serveur systématique du token,
 - séparation des secrets via variables d'environnement,
 - CORS configuré avec origines explicites,
 - messages d'erreur contrôlés côté API,
 - aucune clé secrète exposée dans le frontend.
-

4) Fonctionnalités livrées

4.1 Auth obligatoire

- inscription / connexion / déconnexion,
- persistance de session,
- routes protégées,
- différenciation des accès admin.

4.2 Frontend

- interface responsive,
- navigation par routes,
- états de chargement,
- feedback utilisateur (erreurs/succès).

4.3 Intégration backend

- appels API centralisés,

- gestion des erreurs HTTP,
 - CRUD sur plusieurs ressources,
 - synchronisation frontend ↔ backend.
-

5) Défis rencontrés et solutions

Défi 1 — 401 Token invalide ou expiré

Cause probable : token expiré/caché ou incohérence de flux auth entre pages.

Solutions appliquées :

- retry automatique des requêtes auth après 401,
- rafraîchissement de token via provider central,
- amélioration de la vérification côté backend (tolérance de dérive d'horloge),
- robustesse de la détection des droits admin (fallback par email/token).

Défi 2 — 404 API sur mobile

Cause probable : build frontend pointant vers localhost au lieu d'une URL backend publique.

Solutions appliquées :

- amélioration de la résolution de l'URL API côté frontend,
- utilisation d'une variable d'environnement de production (VITE_API_URL) sur Vercel,
- redéploiement frontend/backend après configuration.

Défi 3 — Reconnaissance du rôle admin

Cause probable : statut admin non résolu de façon fiable dans certains scénarios session.

Solutions appliquées :

- renforcement de la logique backend isUserAdmin,
 - prise en compte du contexte token/email,
 - vérification cohérente des variables admin en environnement.
-

6) Bonnes pratiques appliquées

- séparation claire des responsabilités (routes, controllers, services, middleware),
 - code typé TypeScript,
 - validation des entrées,
 - gestion centralisée de l'accès API frontend,
 - configuration environnementale (.env, .env.example) pour sécurité et portabilité.
-

7) Limites actuelles

- dépendance à la configuration correcte des environnements déployés (Vercel/Clerk),
- peu de tests automatisés end-to-end,

- monitoring production minimal.
-

8) Améliorations futures

1. Ajouter une suite de tests E2E (Playwright/Cypress).
 2. Mettre en place une observabilité (logs structurés + alerting).
 3. Améliorer la gestion hors-ligne et synchronisation différée.
 4. Ajouter une granularité plus fine des permissions (RBAC complet).
 5. Renforcer le pipeline CI/CD (checks sécurité + tests avant déploiement).
-

9) Conclusion

Le Laboratoire 3 a permis de finaliser une application full-stack cohérente, avec frontend React intégré à une API backend existante, authentification sécurisée via Clerk, routes protégées et fonctionnalités CRUD opérationnelles. Le projet est présentable pour l'examen final et démontre la maîtrise des concepts clés du cours : architecture client-serveur, sécurité applicative, et intégration frontend-backend.

Annexes (à compléter avant remise)

- Captures d'écran principales (login, dashboard, CRUD, admin)
- Lien du dépôt GitHub
- Nom et matricule
- URL de déploiement (si disponible)